pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Yield from Price

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Quoted on price, compared on yield

The four bonds, quoted on price on 30 September 2026

Issuer	Coupon %	Years	Quote
Quorvane Energy	5.25	5	99.125
Dunmarrow Rail	6.125	7	100.500
Pellwick Paper	4.75	3	100.875
Halvering Foods	7.50	10	97.750

A dealer shows what you pay. The yield is what makes two quotes comparable.

Today: why yield has no formula, Goal Seek, bisection by hand and in a loop, yield_from_price, pytest.raises, and git restore.

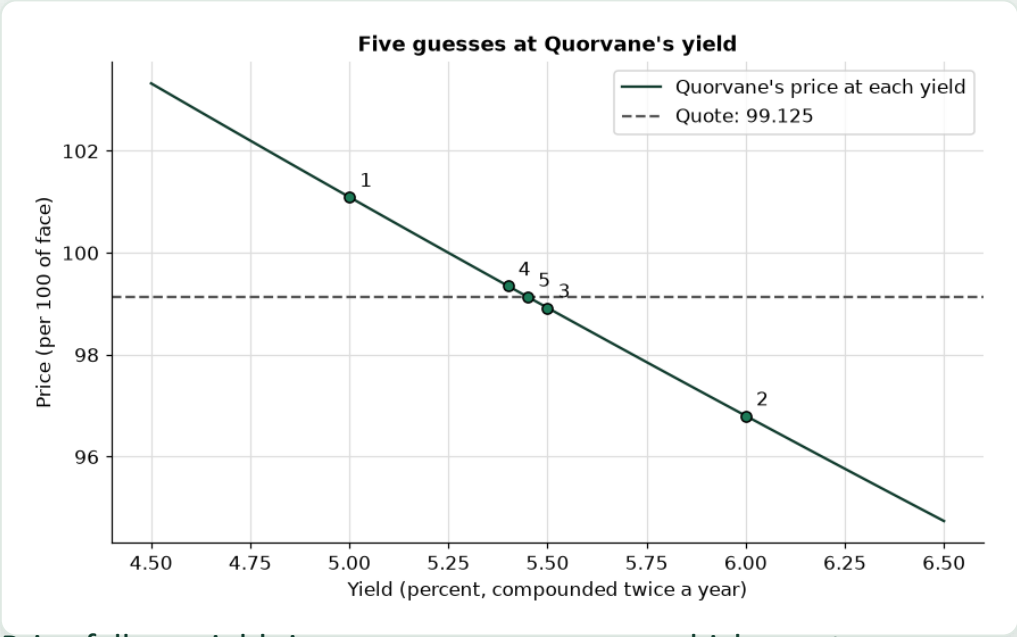
Week 1: price and yield

Day 1	Time value of money
Day 2	Cash flows and price from yield
Day 3	The price-yield relationship
Day 4	**Yield from price**
Day 5	Case study: a new issue calendar

Why yield has no formula

```
# Quorvane: ten half-years, y in percent
99.125 = 2.625 / (1 + y/100/2)^1
       + 2.625 / (1 + y/100/2)^2
       + ...   + 102.625 / (1 + y/100/2)^10
```

Given y, the price is arithmetic. Given the price, y sits inside ten terms, each to a different power. No general formula rearranges it, so a yield is found by search.



Price falls as yield rises, so every guess says which way to go.

Guess a yield, price it, adjust the guess.

Goal Seek: Excel does the guessing

1. Type 5 in A1, a first guess at the yield in percent.

2. In B1:

```
=PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,A1/100,100,2,0)
```

3. Data, What-If Analysis, Goal Seek. Three boxes:

Box	Enter	Meaning
Set cell	B1	the formula
To value	99.125	the quote
By changing cell	A1	the guess

4. Click OK. Excel changes A1 until B1 is close to 99.125.

5.452248

Goal Seek's yield in A1, from a start of 5

+0.000207

B1 minus the quote: close, not exact

Started from 5.25 instead, it stops at 5.452294. YIELD gives 5.452297.

Fine for a quick answer in a sheet. An answer key needs the same answer every time, as close as we say.

Bisection: always guess the middle

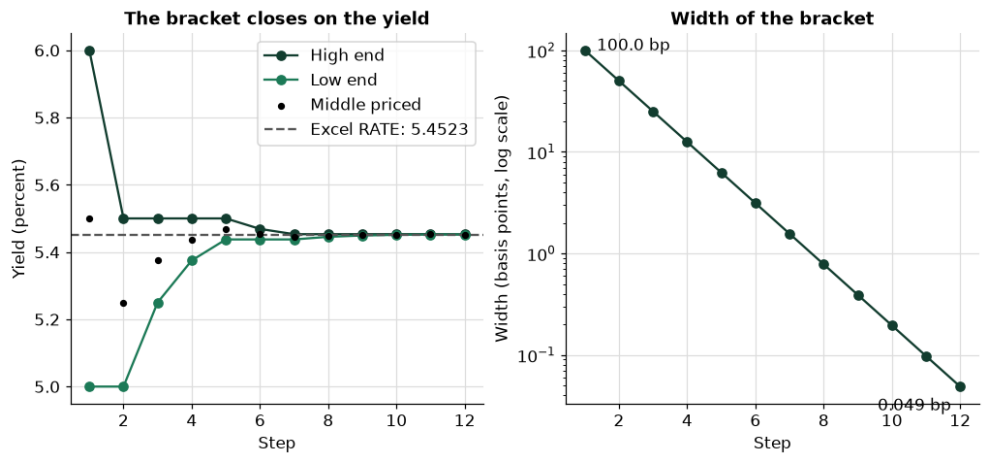
Step	Low	High	Middle	Price	Keep
1	5.000000	6.000000	5.500000	98.9200	lower
2	5.000000	5.500000	5.250000	100.0000	upper
3	5.250000	5.500000	5.375000	99.4583	upper
4	5.375000	5.500000	5.437500	99.1887	upper
5	5.437500	5.500000	5.468750	99.0542	lower
12	5.452148	5.452637	5.452393	99.1246	lower

Steps 6 to 11 left out. Width: 100 bp, 50, 25, ... halved every step.

Price above the quote: the yield is above the middle, keep the upper half. Bisection cannot miss.

Excel side by side (tested in Excel)

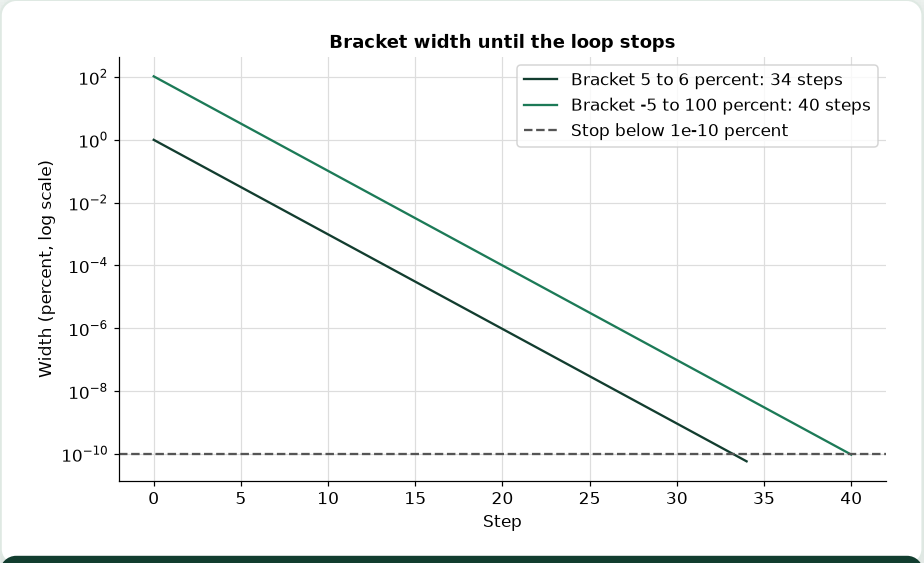
`5 and 6 in A2:B2`	the bracket
`=(A2+B2)/2`	C2, the middle
`=PRICE(...,C2/100,100,2,0)`	D2, price at middle
`=IF(D2>99.125,C2,A2)`	A3, new low
`=IF(D2>99.125,B2,C2)`	B3, new high
`fill A3:D3 down`	C13 5.452393



A while loop: stop when narrow enough

```
low_pct, high_pct = 5.0, 6.0
while high_pct - low_pct > 1e-10:
    mid_pct = (low_pct + high_pct) / 2
    if price_from_yield(5.25, mid_pct, 5) > 99.125:
        low_pct = mid_pct
    else:
        high_pct = mid_pct
yield_pct = (low_pct + high_pct) / 2
```

A while loop repeats while its condition is true. The bracket halves every time, so the condition always ends up false.



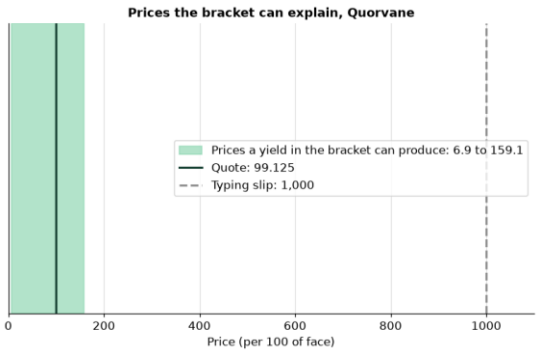
Quorvane at 99.125: 5.452297 percent

A bracket 105 times wider costs about seven more halvings. No good first guess needed.

yield_from_price, with a check that stops

Issuer	Coupon %	Quote	Yield %
Quorvane Energy	5.25	99.125	5.4523
Dunmarrow Rail	6.125	100.500	6.0364
Pellwick Paper	4.75	100.875	4.4353
Halvering Foods	7.50	97.750	7.8286

-5 to 100 percent: prices 6.8931 to 159.0631



Quoted above 100, the yield is below the coupon. Below 100, above it.

Excel side by side (tested in Excel)

`=RATE(2*5,5.25/2,-99.125,100)*2`	0.05452296503527843
`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.25/100,99.125,100,2,0)`	0.054522965032957

pytest.raises: a test that expects an error

Day 2: try and except

```
try:
    bondmath.cash_flows(6.0, 2.3, 2)
except ValueError:
    return
assert False, "accepted 2.3 years"
```

Day 4: pytest.raises

```
with pytest.raises(ValueError):
    bondmath.yield_from_price(
        6.0, 1000.0, 5, 2)
```

The block passes only if the code inside raises ValueError. If the code returns normally, the test fails with DID NOT RAISE.

The check that stops gets a test that stops: a price of 1,000, and a price of 0.01.

Price to yield to price

```
5.40 percent  ->  99.350327  ->  5.400000 percent
```

```
99.125  ->  5.452297 percent  ->  99.125000
```

Quorvane both ways. Each yield comes back inside the final bracket of the search, and each price to within a ten-billionth.

A round trip checks that two functions undo each other. The test against Excel checks that they are right.

11 passed

days 1 to 3, and today's three tests

Today: matches Excel's RATE and YIELD, raises outside the bracket, and the round trip.

Today's Excel, every step tested

Job	Excel (tested)	Returns
Goal Seek	`Set cell B1, To value 99.125, By changing cell A1`	A1 5.452248
Bisection sheet	`=IF(D2>99.125,C2,A2)` and `=IF(D2>99.125,B2,C2)`, filled down	C13 5.452393
Annuity route	`=RATE(2*5,5.25/2,-99.125,100)*2`	0.054523
Bond route	`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.25/100,99.125,100,2,0)`	0.054523
In percent	`=RATE(2*5,5.25/2,-99.125,100)*2*100`	5.452297

RATE returns the rate per period: times 2 for annual, times 100 for percent. The price goes in negative, money paid.

Excel takes decimals: the coupon is written /100, and both RATE and YIELD return a decimal.

The undo button: git restore

Step	Command	What you see
1	a bad edit deletes the bracket check	a price of 1,000 gives -5.0000 percent
2	pytest	1 failed: DID NOT RAISE ValueError
3	git diff bondmath.py	5 lines removed, none added
4	git restore bondmath.py	the file as it was in the last commit
5	pytest	11 passed

git restore discards the uncommitted change for good. Read the diff first.

Before the edit, today's work is committed: Add yield_from_price, tested against Excel RATE and YIELD.

Bring it home: paste, test, commit, restore

1

Your folder

Set-Location to my_bondmath, in your projects folder.

2

Paste

Today's four append cells at the end of your two files, without the %%writefile line.

3

Test

python -m pytest -q: 11 passed, or 14 with the exercises of days 1 to 3.

4

Commit

git diff --stat, git add, git commit with a subject and a body, git log.

5

Undo

Before a commit, a bad edit: git status, git diff, git restore.

Pasting at the end keeps anything of your own. Copy fallen behind? Take the start-of-day files from today's setup cell, and git diff shows what changed.

In Colab: download bondmath.py and test_bondmath.py from the work folder again. They replace yesterday's downloads.

The AI check

The docstring is the prompt. The code that comes back runs, and has one convention error.

1

Docstring

Units, convention, and the Excel formula: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert against Excel's reference rows. Run it: it fails.

→

4

Fix

Find the convention it broke, fix it, run the test again.

Today: yield_from_price returns the rate per period, half the annual yield for a semiannual bond.

Code that runs is not the same as code that is right.

What you can do now

- Say why yield has no formula, and find it by search
- Use Goal Seek, and say how close it gets
- Halve a bracket by hand, in a sheet, and in a while loop
- Solve yield_from_price, RATE, and YIELD; test errors with pytest.raises
- Check with a round trip, and undo a bad edit with git restore

Week 1, Day 5 Case study: a new issue calendar, and a workbook of Excel formulas